# CSE220 Lab3: Perceptron Branch Predictor

by Aaron Vuong, Ashwani Rathee

In this lab, we implemented and evaluated the perceptron branch predictor in the following paper ([Daniel Jimenez, Calvin Lin: Dynamic branch prediction with perceptrons](#)).

Perceptrons are structures that intake multiple inputs and use them to output a single value. This paper used the bits of the history register as its inputs and applied a dot product operation with the perceptron weights to produce a negative or positive value to indicate if the branch should be TAKEN or NOT TAKEN.

## Implementation

Git Repository: https://github.com/Aaron-Vuong/scarab/tree/ash/predictor

| Step | Implemented |
|---|---|
| 1. Hash the branch address to get a perceptron index | `#define BASE_PERCEPTRON_HASH(addr) (addr % BASE_PERCEPTRON_ENTRIES)`<br>`uns32 index = BASE_PERCEPTRON_HASH(addr);` |
| 2. The indexed perceptron is fetched | `const auto& perceptron_state = perceptron_hist.at(proc_id).perceptron_state_all_cores[index];` |
| 3. Compute $y$ | `for(int i = 0; i < BASE_HIST_LENGTH; ++i) {`<br>`  int history_bit = ((hist >> (32 - i)) & 0x1) ? 1 : -1;`<br>`  // Bias weight is just the weight.`<br>`  if (i == 0) {`<br>`  prediction_score += perceptron_state.weights[i];`<br>`  continue;`<br>`  }`<br>`  prediction_score += ((history_bit * perceptron_state.weights[i]));`<br>`}` |
| 4. Send the prediction of the branch. | `uns8 pred = (prediction_score >= 0.0f) ? 1 : 0;`<br>`return pred;` |
| 5. Train the perceptron after the branch outcome is known. | `if(!(base_perceptron_output <= 0 && t <= 0) \|\| (abs(base_perceptron_output) <= PERCEPTRON_THRESHOLD)) {`<br>`  for(int i = 0; i < BASE_HIST_LENGTH; ++i) {`<br>`  int history_bit = ((hist >> (32 - i)) & 0x1) ? 1 : -1;`<br>`  // Bias weight is just the weight.`<br>`  if (i == 0) {`<br>`  perceptron_state.weights[i] += t;`<br>`  } else {` |

| | |
|---|---|
| | `perceptron_state.weights[i] += history_bit * t;`<br>`}`<br>`}` |
| 6. The perceptron is written back into the table. | The perceptron is a pointer so it is already modified in the table. |

# Evaluation & Results

We used the following configurations to gather data about our final implementation, where we abbreviated each of the parameters to the following.

- **history_length** to **hl**
- **num_perceptrons_in_table** to **bpr**

Experiments Ran:

| 1. Optimal History Length | 2. Optimal # of Perceptrons | 3. Branch Predictor Comparison |
|---|---|---|
| -base_hist_length [1-32]<br>-base_perceptron_rows 2048 | -base_hist_length 8<br>-base_perceptron_rows [1-32768] | -bp_mech perceptron<br>-bp_mech gshare<br>-bp_mech tage64k |

There were two primary experiments that we ran.

- Adjust the number of history length bits to the most optimal value.
- Adjust the number of perceptrons in the perceptron table to the most optimal value, for the optimal history length.

## 1. Finding the optimal history length

From the graph, we can see that with 2048 perceptrons, the optimal history length to look at is ~8 bits of the most recent history. This is in line with the paper itself, which recommended 7 or 9 bits (1 bit for the sign), with 8 + 1 sign bit being the most optimal history length.

We actually see a decrease in overall IPC and Branch Mispredictions when moving from 8 bits to 16/32 bits.

- AVG IPC: **1.58478** (8bits) -> **1.58217** (32bits)
- AVG Mispredict %: **5.46334%** (8bits) -> **5.6417%** (32bits)

We think that if we look at too many values, since we are just referring to the global history, it can mislead our perceptron into relying too much on old information to make predictions about the current state.

## 2. Finding the optimal number of perceptrons

Now that we have our optimal value of 8 bits, we can also look at how many perceptrons to have in our table of perceptrons. We collected another sweep of values, going from just 1 perceptron to 32768 perceptrons.

The maximum value of 32768 perceptrons has the most performance, maxing out at **1.59391** IPC as the average of all benchmarks. Also, we see an asymptotic curve as we approach the higher values of perceptrons, where jumping from 1 to 8 perceptrons had the largest effect on IPC.

Increasing the number of perceptrons addresses the main problem of aliasing and interference between different branches. As we increase the number of perceptrons, each branch can get its own predictor and weights that accurately match its pattern. However, as we approach the number of unique branch addresses in the program with the number of perceptrons, we will have removed all interference and there can be no more performance gain to be made which we can observe in our graph.

## 3. Comparison to other branch predictors

In our final experiment/graph, we ran other branch predictors with the default PARAMS.sunny_cove file. While we didn't beat the other predictors, we were at least able to approach near the efficiency of gshare and tage64k, having **5.3866%** average mispredict ratio to gshare's **3.14337%** average mispredict ratio.

One thing that could explain the differences in perceptron behavior versus the paper is the benchmarks themselves. Perceptrons can't learn non-linearly separable functions with 100% accuracy while gshare can (given enough time). If the benchmarks have changed between SPEC2000 and SPEC2017 to contain more of these types of functions, it could explain the large spikes in mispredictions on certain benchmarks for the perceptron predictor.

# Lab 3: Perceptron Branch Predictor

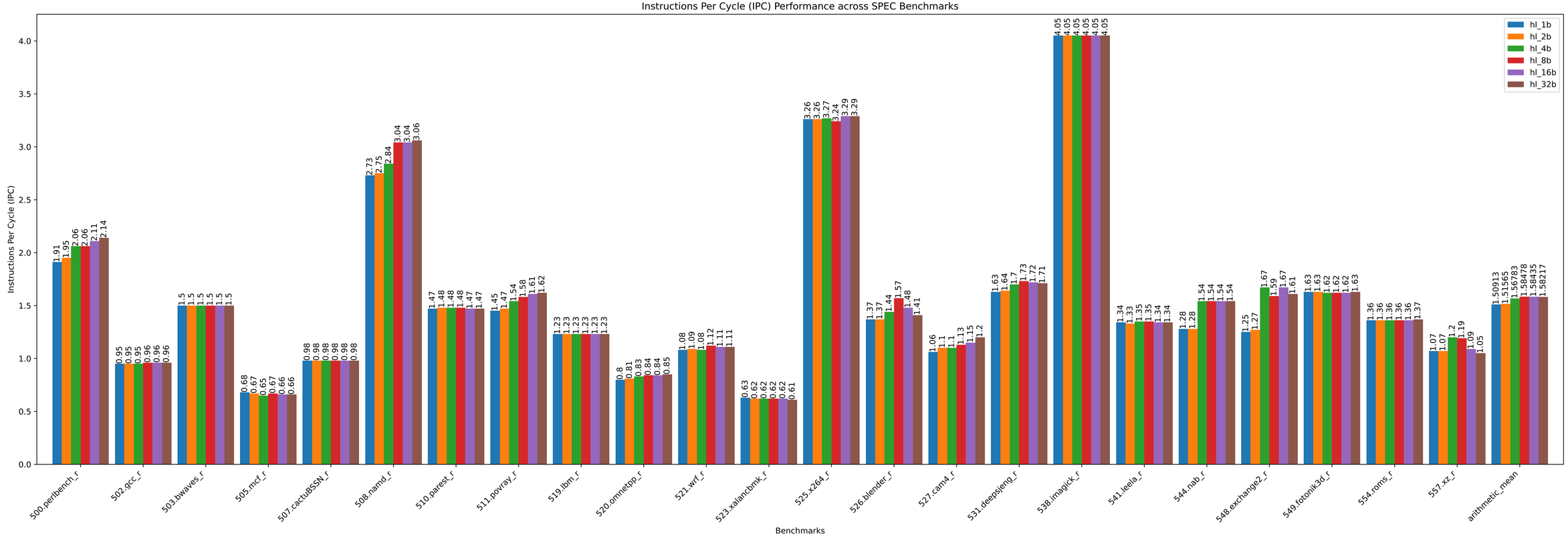
## Experiment 1: Optimal History Length

### JSON Configuration

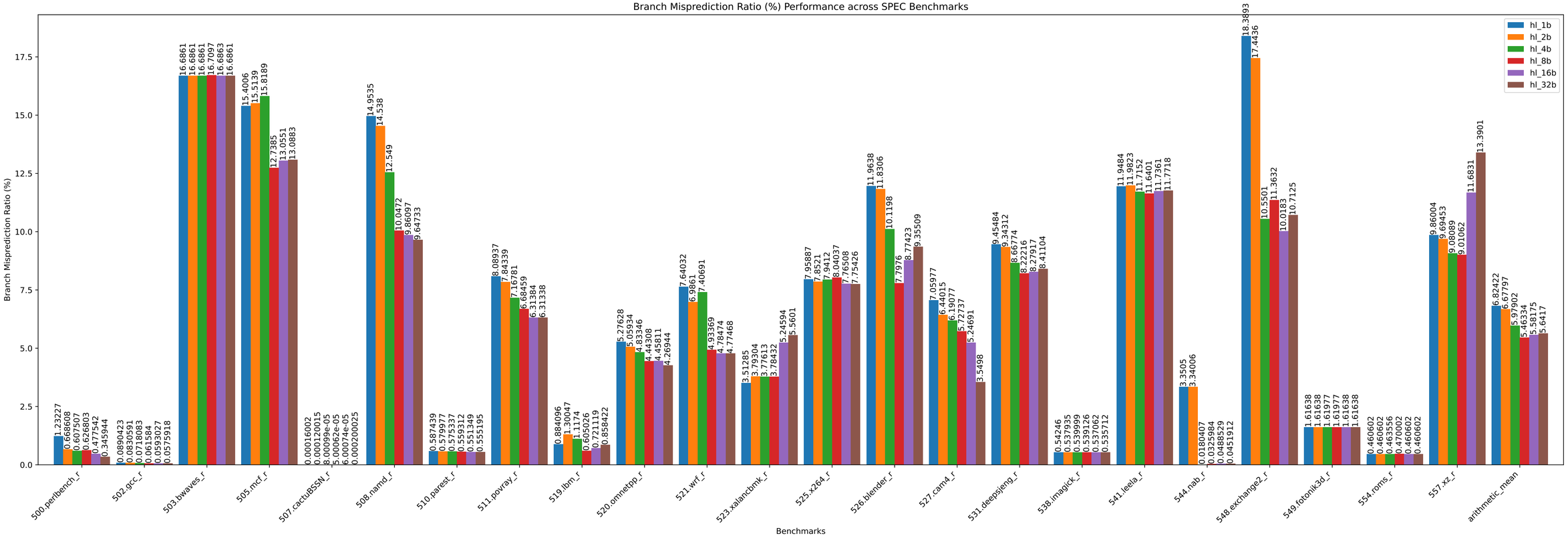

```
{
   "architecture": "sunny_cove",
   "workloads_list": [
      "500.perlbench_r",
      "502.gcc_r",
      "503.bwaves_r",
      "505.mcf_r",
      "507.cactuBSSN_r",
      "508.namd_r",
      "510.parest_r",
      "511.povray_r",
      "519.lbm_r",
      "520.omnetpp_r",
      "521.wrf_r",
      "523.xalancbmk_r",
      "525.x264_r",
      "526.blender_r",
      "527.cam4_r",
      "531.deepsjeng_r",
      "538.imagick_r",
      "541.leela_r",
      "544.nab_r",
      "548.exchange2_r",
      "549.fotonik3d_r",
      "554.roms_r",
      "557.xz_r"
   ],
   "experiment": "lab3",
   "configurations": {
      "hl_1b": "--bp_mech perceptron --base_hist_length 1 --base_perceptron_rows 2048",
      "hl_2b": "--bp_mech perceptron --base_hist_length 2 --base_perceptron_rows 2048",
      "hl_4b": "--bp_mech perceptron --base_hist_length 4 --base_perceptron_rows 2048",
      "hl_8b": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 2048",
      "hl_16b": "--bp_mech perceptron --base_hist_length 16 --base_perceptron_rows 2048",
      "hl_32b": "--bp_mech perceptron --base_hist_length 32 --base_perceptron_rows 2048"
   }
}
```

Lab 3: Perceptron Branch Predictor

Instructions Per Cycle (IPC) Performance across SPEC Benchmarks




Branch Misprediction Ratio (%) Performance across SPEC Benchmarks

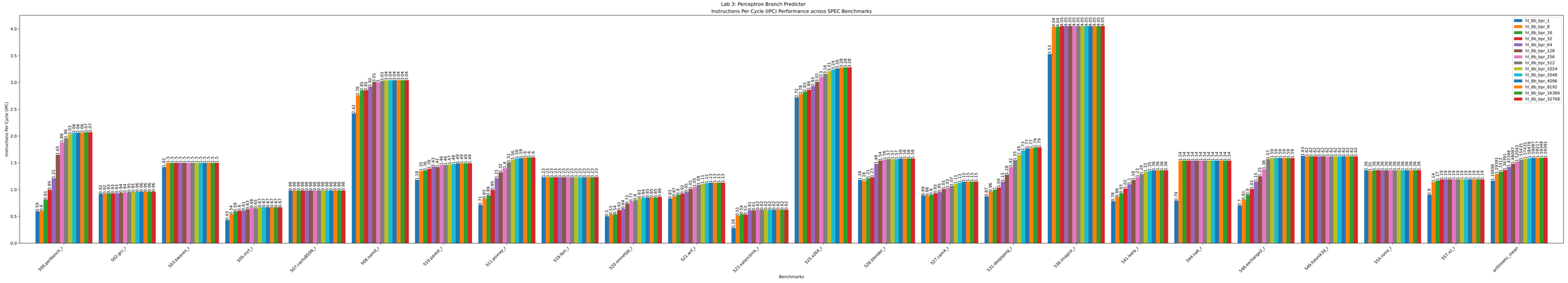


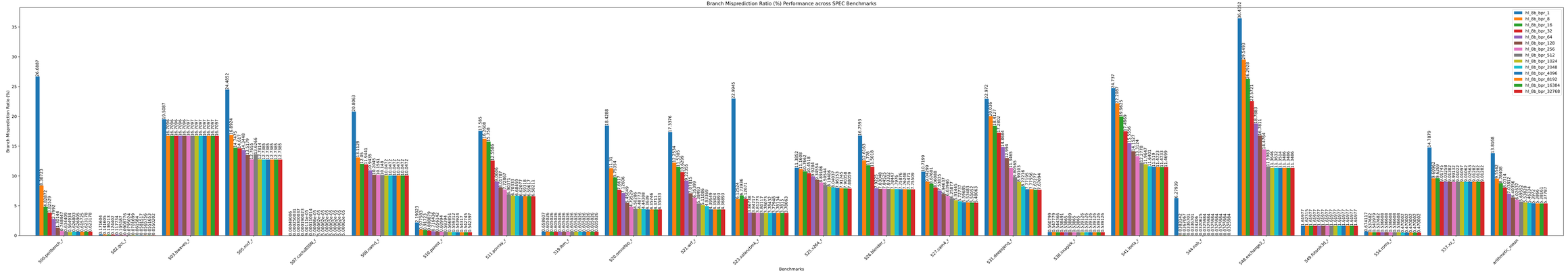
# Lab 3: Perceptron Branch Predictor

## Experiment 2: Optimal # of Perceptrons

## JSON Configuration


```
{
   "architecture": "sunny_cove",
   "workloads_list": [
      "500.perlbench_r",
      "502.gcc_r",
      "503.bwaves_r",
      "505.mcf_r",
      "507.cactuBSSN_r",
      "508.namd_r",
      "510.parest_r",
      "511.povray_r",
      "519.lbm_r",
      "520.omnetpp_r",
      "521.wrf_r",
      "523.xalancbmk_r",
      "525.x264_r",
      "526.blender_r",
      "527.cam4_r",
      "531.deepsjeng_r",
      "538.imagick_r",
      "541.leela_r",
      "544.nab_r",
      "548.exchange2_r",
      "549.fotonik3d_r",
      "554.roms_r",
      "557.xz_r"
   ],
   "experiment": "lab3",
   "configurations": {
      "hl_8b_bpr_1": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 1",
      "hl_8b_bpr_8": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 8",
      "hl_8b_bpr_16": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 16",
      "hl_8b_bpr_32": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 32",
      "hl_8b_bpr_64": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 64",
      "hl_8b_bpr_128": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 128",
      "hl_8b_bpr_256": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 256",
      "hl_8b_bpr_512": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 512",
      "hl_8b_bpr_1024": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 1024",
      "hl_8b_bpr_2048": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 2048",
      "hl_8b_bpr_4096": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 4096",
      "hl_8b_bpr_8192": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 8192",
      "hl_8b_bpr_16384": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 16384",
      "hl_8b_bpr_32768": "--bp_mech perceptron --base_hist_length 8 --base_perceptron_rows 32768"
   }
}
```


Lab 3: Perceptron Branch Predictor
Instructions Per Cycle (IPC) Performance across SPEC Benchmarks
Instructions Per Cycle (IPC)
Benchmarks
hl_8b_bpr_1
hl_8b_bpr_8
hl_8b_bpr_16
hl_8b_bpr_32
hl_8b_bpr_64
hl_8b_bpr_128
hl_8b_bpr_256
hl_8b_bpr_512
hl_8b_bpr_1024
hl_8b_bpr_2048
hl_8b_bpr_4096
hl_8b_bpr_8192
hl_8b_bpr_16384
hl_8b_bpr_32768
500.perlbench_r
502.gcc_r
503.bwaves_r
505.mcf_r
507.cactuBSSN_r
508.namd_r
510.parest_r
511.povray_r
519.lbm_r
520.omnetpp_r
521.wrf_r
523.xalancbmk_r
525.x264_r
526.blender_r
527.cam4_r
531.deepsjeng_r
538.imagick_r
541.leela_r
544.nab_r
548.exchange2_r
549.fotonik3d_r
554.roms_r
557.xz_r
arithmetic_mean


Branch Misprediction Ratio (%) Performance across SPEC Benchmarks
Branch Misprediction Ratio (%)
Benchmarks
hl_8b_bpr_1
hl_8b_bpr_8
hl_8b_bpr_16
hl_8b_bpr_32
hl_8b_bpr_64
hl_8b_bpr_128
hl_8b_bpr_256
hl_8b_bpr_512
hl_8b_bpr_1024
hl_8b_bpr_2048
hl_8b_bpr_4096
hl_8b_bpr_8192
hl_8b_bpr_16384
hl_8b_bpr_32768
500.perlbench_r
502.gcc_r
503.bwaves_r
505.mcf_r
507.cactuBSSN_r
508.namd_r
510.parest_r
511.povray_r
519.lbm_r
520.omnetpp_r
521.wrf_r
523.xalancbmk_r
525.x264_r
526.blender_r
527.cam4_r
531.deepsjeng_r
538.imagick_r
541.leela_r
544.nab_r
548.exchange2_r
549.fotonik3d_r
554.roms_r
557.xz_r
arithmetic_mean

# Lab 3: Perceptron Branch Predictor

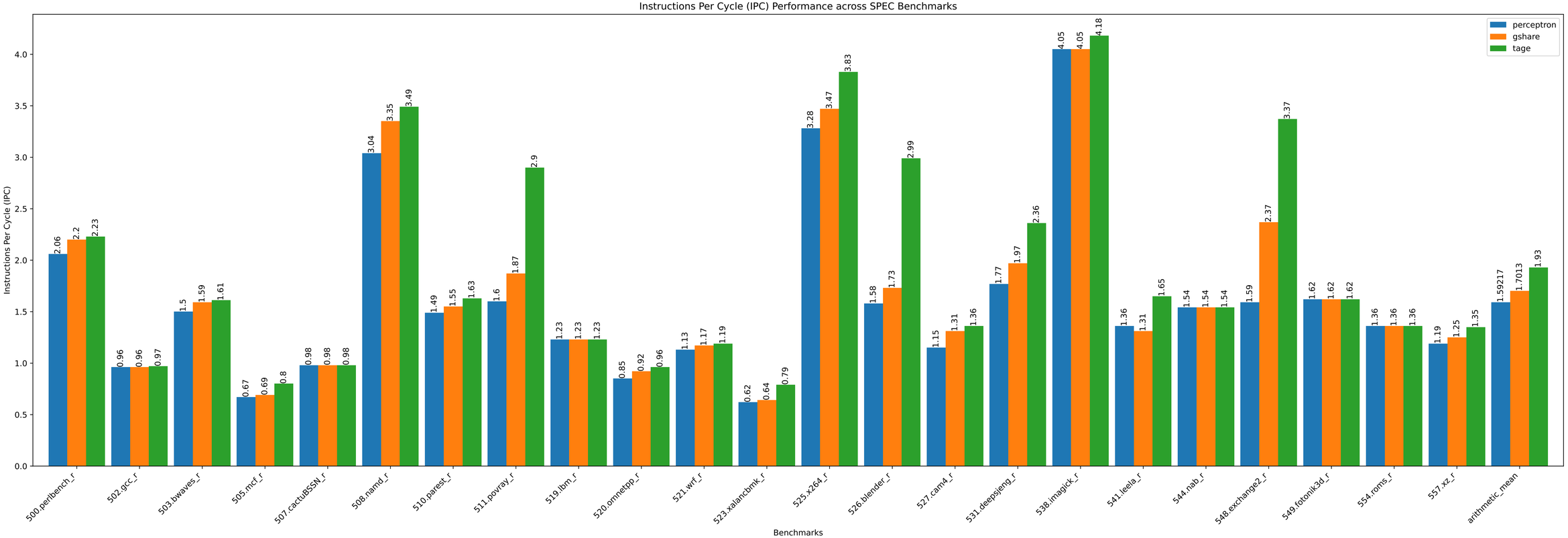
## Experiment 3: Branch Predictor Comparison

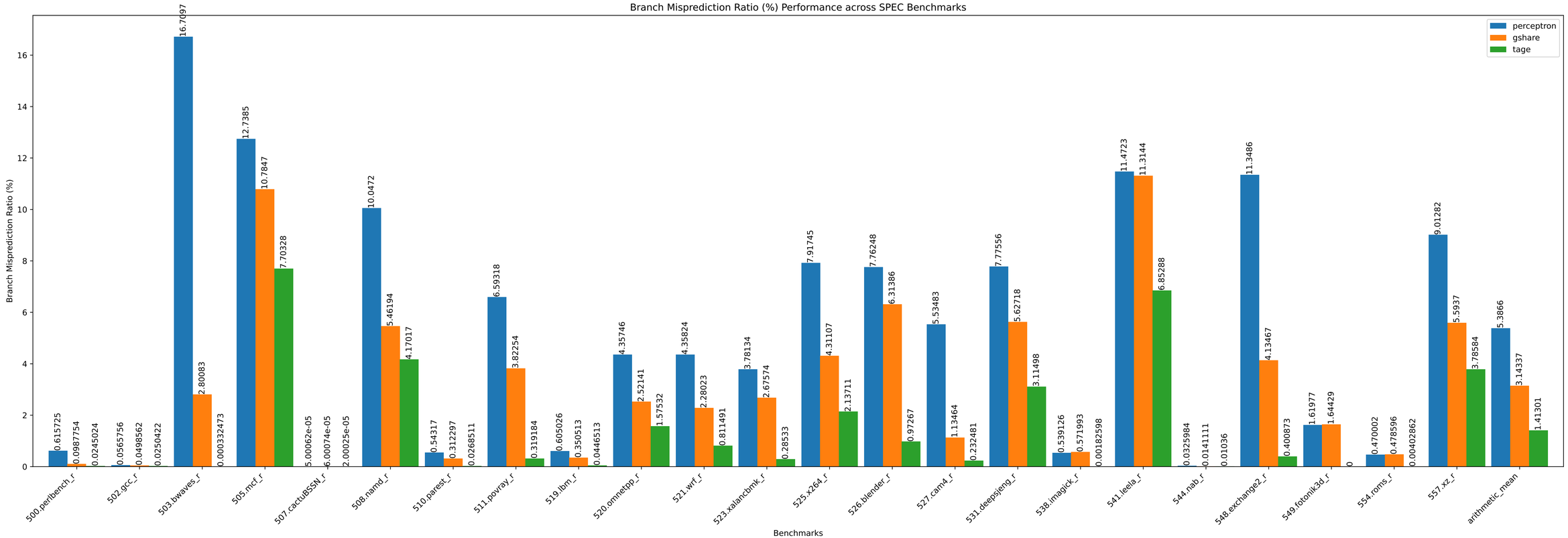
### JSON Configuration


```
{
   "architecture": "sunny_cove",
   "workloads_list": [
      "500.perlbench_r",
      "502.gcc_r",
      "503.bwaves_r",
      "505.mcf_r",
      "507.cactuBSSN_r",
      "508.namd_r",
      "510.parest_r",
      "511.povray_r",
      "519.lbm_r",
      "520.omnetpp_r",
      "521.wrf_r",
      "523.xalancbmk_r",
      "525.x264_r",
      "526.blender_r",
      "527.cam4_r",
      "531.deepsjeng_r",
      "538.imagick_r",
      "541.leela_r",
      "544.nab_r",
      "548.exchange2_r",
      "549.fotonik3d_r",
      "554.roms_r",
      "557.xz_r"
   ],
   "experiment": "lab3",
   "configurations": {
      "perceptron": "--bp_mech perceptron",
      "gshare": "--bp_mech gshare",
      "tage": "--bp_mech tage64k"
   }
}
```

Lab 3: Perceptron Branch Predictor

Instructions Per Cycle (IPC) Performance across SPEC Benchmarks




Branch Misprediction Ratio (%) Performance across SPEC Benchmarks